

PRINCIPLES OF PROGRAMMING LANGUAGES • LESSON 01

Recursion

A study of self-reference, execution state, and termination

Recursion expresses a computation in terms of smaller or otherwise controlled instances of the same problem. This guide develops the idea from a mathematical definition to concrete runtime behavior in Python, JavaScript, and C++.

The emphasis is on explanation: what is suspended at each call, what is shared, what makes execution stop, and what a language implementation must preserve.

Course alignment

Syllabus module	Connection to this lesson
1. Foundations of Programming Languages	Recursive definitions, control flow, syntax trees, operational meaning, and correctness.
2. Programming Languages and Paradigms	Functional decomposition, imperative state, memoization, and graph traversal.
3. Design Dimensions of Sub Programs	Parameters, local bindings, activation records, closures, references, and return behavior.

Learning outcomes

- Define recursion and distinguish a base case from a progress argument.
- Trace descent, suspended work, return values, and active call depth.
- Explain the difference between a call stack, a call tree, and a cache.
- Analyze three complete programs, including their boundaries and costs.
- Choose between recursion, iteration, and an explicit stack using evidence.

Reading route

Core concept (pp. 2-3) • Runtime mechanics (p. 4) • Python factorial (pp. 5-6) • JavaScript memoization (pp. 7-8) • C++ cycle detection (pp. 9-10) • Design synthesis (p. 11) • Assessment (p. 12) • References (p. 13).

Prerequisites: functions, conditionals, variables, basic arrays/lists, and elementary Big-O notation. Examples target Python 3, modern JavaScript with BigInt, and C++17 or later.

1 • INTRODUCTION & CORE CONCEPT

What recursion means

An academic definition

Recursion is a method of definition or computation in which an entity is characterized using instances of itself. In a recursive subprogram, evaluation may invoke that subprogram again, directly or through other subprograms. Each invocation has its own logical execution state, even though the invocations share the same function definition.

A recursive definition must distinguish cases that can be resolved directly from cases that depend on further work. For factorial over nonnegative integers, the defining equations are $F(0) = 1$ and $F(n) = n \times F(n - 1)$ for $n > 0$. The first equation supplies a known result; the second reduces the input while specifying how to combine the smaller result.

Why languages support recursion

Recursive structure occurs naturally in syntax, data, and algorithms. An arithmetic expression can contain smaller expressions. A directory contains files and subdirectories. A tree consists of a root and smaller trees. A grammar may define a construct in terms of that construct nested within another one. Recursive functions let program structure follow these definitions closely.

- **Compositionality:** a result can be assembled from results for constituent parts.
- **Local reasoning:** the programmer specifies one case and relies on a clear contract for each recursive call.
- **Abstraction:** the runtime manages suspended calls instead of requiring the programmer to explicitly store every return point.
- **General subprogram semantics:** a function call can target the currently executing function when the language's naming and call rules permit it.

Three obligations

Base case: identify an input whose result is available without another recursive call. **Progress:** show that recursive calls approach a base case or are restricted by a finite visitation policy. **Combination:** explain how returned results establish the caller's result. A base case written in the source is insufficient if execution can never reach it.

Recursion is not textual copying

Calling a function again does not paste another copy of its source into the program. The same compiled or interpreted function representation is invoked with a new logical context. Separate calls can have different arguments, local bindings, and pending operations. This separation is what makes the return phase meaningful.

1 • INTRODUCTION & CORE CONCEPT

Forms, contracts, and proofs

Useful categories

Form	Defining characteristic	Typical use
Direct	A function calls itself.	Factorial; tree height.
Indirect / mutual	Calls form a cycle across functions.	Mutually dependent grammar rules.
Linear	At most one recursive call is made by an activation.	A countdown or list traversal.
Branching	An activation can make multiple recursive calls.	Divide-and-conquer; tree traversal.
Tail	The call's result is returned without pending combination work.	An accumulator-style recurrence.

These categories overlap. A function can be both directly recursive and tail recursive. A function with two call sites does not necessarily create exponentially many distinct subproblems: its actual inputs and reuse policy determine the work.

A recursive contract

A contract identifies the input domain, promised result, side effects, and relevant failure conditions. For factorial: the input is a nonnegative built-in integer; the result is its factorial if sufficient execution resources are available; invalid input raises an exception. The resource qualification separates mathematical termination from practical executability.

Proof by induction

For factorial, prove the base case $F(0) = 1$. For the inductive step, assume the function correctly computes $F(n - 1)$ for a positive integer n . Multiplying that result by n gives the defining value of $F(n)$. This establishes correctness for each value, provided the recursive evaluation terminates and the operations have the required arithmetic semantics.

A well-founded progress measure

A **ranking function** maps program states into an order in which an infinite descending chain is impossible. For factorial, use the nonnegative integer n : every recursive call decreases it by one until zero. For graph traversal, numerical arguments may not decrease; a finite-state visitation rule instead limits which vertices may be expanded.

Partial correctness means that a terminating execution produces the specified result. **Total correctness** additionally establishes termination. Neither proves that a real machine has enough stack or memory for every mathematically valid input.

Checkpoint

Why does replacing $n - 1$ with $n + 1$ break factorial? The base case remains present, but positive inputs move away from it. The defect is a failed progress argument, not the absence of an if statement.

2 • UNDER THE HOOD

Calls, frames, and suspended work

From source to execution

Tokenization recognizes keywords, names, delimiters, and operators. Parsing organizes them into a syntax tree. A recursive call is generally represented as an ordinary call expression; no special 'recursion instruction' is needed at the source-language level. Python's AST provides function-definition, call, conditional, and return nodes [1]. A same-name call can suggest direct recursion, but name rebinding and indirect calls mean syntax alone does not completely determine runtime call behavior.

Activation records: a conceptual model

A function activation needs enough information to continue execution: arguments or parameter bindings, local state, a return destination, and pending work. For $F(3)$, evaluation must retain the multiplications by 3 and 2 while smaller calls run. Returning from $F(0)$ supplies a result to its caller; that caller completes its own work and returns again.

Suspended chain: $F(3)$ waits to multiply by 3; $F(2)$ waits to multiply by 2; $F(1)$ waits to multiply by 1; $F(0)$ returns 1 immediately. Completion follows the reverse order of descent.

Call stack versus call tree

The **call stack** describes currently active, nested calls. The **call tree** describes all call occurrences during an execution. A branching algorithm may have exponentially many total calls while only a linear number are active at once. Total work and maximum active depth are different quantities.

Language semantics versus implementation

ECMAScript specifies execution contexts and lexical environments as abstract mechanisms; it does not require a particular physical frame layout [3]. Likewise, conceptual Python activations should not be equated one-for-one with native C stack frames across all implementations. C++ implementations commonly use native call stacks, but optimizations may alter storage while preserving observable behavior.

Local scope is not a claim about a physical memory address. Objects can reside in managed or dynamically allocated storage while a call retains references to them. In C++, automatic local objects have scope-related lifetimes [6]; a reference parameter can provide access to a container whose storage outlives the call.

Depth limits and failure

Python exposes an interpreter recursion limit through `sys.getrecursionlimit()`; excessive depth raises `RecursionError`. Raising the limit is not a termination fix [2]. JavaScript engines have practical depth limits with engine-dependent errors [5]. C++ provides no portable promise that exhausting a native stack will produce a catchable standard exception.

3 • CODE EXAMPLES / SIMPLE

Python: recursive factorial

This example makes the domain explicit and exposes pending multiplication. Run it as a standalone Python 3 program.

PYTHON 3 • Simple / reduction of an integer

```
def factorial(n):
    """Return n! for a nonnegative built-in integer."""
    if type(n) is not int:
        raise TypeError("n must be an integer")
    if n < 0:
        raise ValueError("n must be nonnegative")
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

In-depth reasoning: execution, statement by statement

The function definition creates a callable associated with the name `factorial`. Calling `factorial(5)` binds `n` to 5 in a new activation. The exact-type check deliberately rejects booleans and integer subclasses as part of this example's contract; it is stricter than many general-purpose numerical APIs. The nonnegative check prevents a descending sequence of negative inputs from missing zero forever.

At `n = 5`, the base test is false. The return expression cannot finish until `factorial(4)` supplies its result. Each successive call has its own `n` binding. The caller's binding is not decremented in place: the expression `n - 1` provides a value for the next invocation. At `n = 0`, the function returns 1 without making another call.

Return then proceeds outward. Each waiting activation multiplies its own `n` by the returned value and passes the product to its caller. The final print displays **120**. The lexical function definition is shared, but the logical activations and their pending operations are distinct.

Cross-language interpretation

Python's built-in integers support arbitrary precision [8], so this example has no fixed-width integer overflow boundary. JavaScript Number arithmetic would eventually lose integer precision; `BigInt` is an alternative. Ordinary C++ integer types have finite ranges, so a factorial implementation must choose a suitable representation and check arithmetic limits. A terminating recurrence alone does not guarantee a representable result.

3 • PYTHON / DETAILED ANALYSIS

Descent, return, and correctness

The complete call-and-return trace

Call	Next action during descent	Returned value
factorial(5)	Wait for factorial(4); multiply by 5.	120
factorial(4)	Wait for factorial(3); multiply by 4.	24
factorial(3)	Wait for factorial(2); multiply by 3.	6
factorial(2)	Wait for factorial(1); multiply by 2.	2
factorial(1)	Wait for factorial(0); multiply by 1.	1
factorial(0)	Base case: return immediately.	1

Read the call column downward for descent and the returned values upward for completion. There are six factorial activations at maximum depth for input 5, counting the base call. The function is **not tail recursive**: multiplication remains after every non-base recursive call.

Formal reasoning

Base: for $n = 0$, the result is 1, which defines $0!$. **Step:** for $n > 0$, suppose $\text{factorial}(n - 1)$ returns $(n - 1)!$. The caller returns $n \times (n - 1)! = n!$. **Termination:** n remains nonnegative and decreases by one on each recursive call until zero. Validation ensures this argument applies to the accepted input domain.

Cost model

There are $n + 1$ factorial calls and n multiplications. Under a unit-cost arithmetic model, time is $O(n)$ and active control state is $O(n)$. This is not a complete bit-complexity bound: factorial results grow rapidly, so the costs of multiplication and storing large integers increase. ' $O(n)$ stack' also counts logical frames; it is not an exact byte measurement of a particular Python interpreter.

Boundary and failure analysis

- **0:** returns 1 immediately. **1:** makes one recursive call and returns 1.
- **-1:** raises `ValueError` before recursion. **2.5 or True:** raises `TypeError` under the stated exact-type policy.
- **Very large valid input:** can exceed practical recursion depth even though the recurrence is mathematically terminating [2].

How iteration would change the mechanism

An iterative accumulator can perform the same multiplications while retaining only a bounded number of scalar bindings. It removes the linear chain of pending returns, although the large result still occupies growing storage. Recursion is valuable here as a model of inductive reasoning, not as a claim that recursive factorial is the most robust production implementation.

3 • CODE EXAMPLES / INTERMEDIATE

JavaScript: memoized Fibonacci

Fibonacci illustrates overlapping subproblems: $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n - 1) + F(n - 2)$. The cache avoids recomputing a result already obtained.

JAVASCRIPT • Intermediate / branching with a shared cache

```
function fibonacci(n) {
  if (!Number.isSafeInteger(n) || n < 0) {
    throw new RangeError("n must be a nonnegative safe integer");
  }

  const memo = new Map([[0, 0n], [1, 1n]]);

  function solve(k) {
    if (memo.has(k)) {
      return memo.get(k);
    }

    const left = solve(k - 1);
    const right = solve(k - 2);
    const value = left + right;
    memo.set(k, value);
    return value;
  }

  return solve(n);
}

console.log(fibonacci(10).toString());
console.log(fibonacci(50).toString());
```

In-depth reasoning: calls and shared state

The outer call validates n and creates a new `Map`. Keys are `Number` indices; values are `BigInt` results. The nested `solve` function accesses `memo` through its lexical environment. Each `solve` activation has a distinct `k` binding, but all calls in this invocation share the same cache. The `const` binding prevents reassignment of `memo`; it does not prevent `Map` updates.

A cache hit returns immediately. A miss computes the smaller left result, then the smaller right result, adds them, stores the answer, and returns it. The separate statements make evaluation order explicit. The outputs are **55** and **12586269025**, on separate lines. Calling `toString()` produces decimal text without a `BigInt` literal suffix.

3 • JAVASCRIPT / DETAILED ANALYSIS

What memoization changes

Trace solve(5), starting from the two base entries

Stage	What happens	New cache result
Descent	solve(5), solve(4), solve(3), solve(2) are misses.	None yet
Resolve 2	solve(1) and solve(0) hit the base entries.	2 -> 1n
Resolve 3	Reuse the result for 2 and the cached result for 1.	3 -> 2n
Resolve 4	Reuse 3; solve(2) is now a cache hit.	4 -> 3n
Resolve 5	Reuse 4; solve(3) is now a cache hit.	5 -> 5n

Return behavior and correctness

A cached entry represents a completed result, never a partially computed one. Inductively, the initial entries are correct. When solve(k) finishes, both smaller results are correct, so their sum is $F(k)$, and storing it preserves the cache invariant. The nonnegative index decreases on recursive descent until a cached base is reached. Memoization improves reuse; the decreasing argument establishes termination.

Arithmetic and cross-language differences

BigInt represents arbitrary-size integers subject to available resources [4]. Both operands in left + right are BigInt; mixing a Number with a BigInt in this addition would be an error. Python can use its built-in int for comparable integer results. C++ needs explicit attention to the chosen numeric type or an arbitrary-precision library. Here, n remains a Number used only as a validated integer index.

Time, memory, and depth

Without reuse, the usual Fibonacci recurrence expands repeated subproblems and has exponential call growth. Here each non-base index is computed once: $O(n)$ additions and $O(n)$ stored entries, assuming expected constant-time map operations. Maximum active depth is still $O(n)$. The initial descent has not disappeared; a large accepted index can still exceed an engine's call-depth limit [5].

Unit-cost $O(n)$ is not the bit-complexity bound. $F(k)$ contains $O(k)$ bits, so storing all results through n requires $O(n^2)$ bits, and their additions have growing costs. The two top-level calls allocate separate caches; the second does not reuse the first call's results.

Lifetime and optimization

Once fibonacci returns and no live references retain its nested function or cache, the cache is eligible for garbage collection; collection timing is implementation-dependent. The recursive calls inside solve are not tail calls because addition and cache insertion remain. Tail-call optimization cannot be assumed as a remedy for this program's depth.

3 • CODE EXAMPLES / ADVANCED EDGE CASE

C++: recursion over a cyclic graph

A directed graph need not be a tree. Reaching a vertex currently on the active DFS path proves a directed cycle; reaching a completed vertex does not.

C++17 • Advanced / DFS with explicit visitation states

```
#include <cstdint>
#include <iostream>
#include <vector>

enum class State { unseen, active, done };
using Graph = std::vector<std::vector<std::size_t>>;

bool visit(std::size_t u, const Graph& graph,
          std::vector<State>& state) {
    if (state.at(u) == State::active) {
        return true;
    }
    if (state.at(u) == State::done) {
        return false;
    }
    state.at(u) = State::active;
    for (std::size_t v : graph.at(u)) {
        if (visit(v, graph, state)) {
            return true;
        }
    }
    state.at(u) = State::done;
    return false;
}

bool has_cycle(const Graph& graph) {
    std::vector<State> state(graph.size(), State::unseen);
    for (std::size_t u = 0; u < graph.size(); ++u) {
        if (state[u] == State::unseen && visit(u, graph, state)) {
            return true;
        }
    }
    return false;
}

int main() {
    const Graph cyclic{{1}, {2}, {0}, {}};
    const Graph acyclic{{1, 2}, {3}, {3}, {}};
    std::cout << std::boolalpha;
    std::cout << has_cycle(cyclic) << '\n';
    std::cout << has_cycle(acyclic) << '\n';
}
```

Runtime reasoning. The outputs are true, then false. Each `has_cycle` call allocates fresh state. A `visit` call receives `u` by value and shares `graph/state` by reference. It returns immediately for active or done vertices; otherwise it marks `u` active, explores outgoing edges, and marks `u` done only after all those calls finish without finding a cycle.

3 • C++ / DETAILED ANALYSIS

Cycles, references, and finite progress

Follow the two executions

In the cyclic graph, `visit(0)` marks 0 active and calls `visit(1)`. That call activates 1 and calls `visit(2)`. Vertex 2 points to 0; the next `visit(0)` sees an active vertex and returns true. The true value propagates through the waiting calls. Some states remain active because of the early exit, but the search ends and the local state vector is discarded. They are not reused by another search.

In the acyclic graph, DFS explores `0 -> 1 -> 3`, completes 3, then completes 1. It next explores `0 -> 2 -> 3`. Vertex 3 is done, so this second encounter returns false. Shared descendants are not cycles. A single 'seen' flag would fail to distinguish this case from an edge back to an active ancestor.

Memory and parameter mechanics

The `const Graph&` parameter avoids copying the adjacency lists and prevents modification through that parameter. The state reference permits all nested calls to update one visitation table. Each activation keeps its own `u` and its own traversal position through the current vertex's outgoing edges. References share objects; they do not make all local variables shared.

The state vector owns its element storage and releases it when destroyed. Normal return ends the lifetimes of local automatic objects according to C++ scope rules [6, 7]. A deep native call chain can still exhaust execution resources. This algorithm is not tail recursive: after a successful child-free exploration, the caller must continue its edge loop and eventually mark its own vertex done.

Correctness and termination

During an unfinished depth-first path, active marks identify its vertices. An edge to an active vertex closes a directed path back to an ancestor. A done vertex has already had all outgoing edges checked without discovering a cycle. The outer loop starts searches in every still-unseen component, so disconnected cycles are included.

For a finite, unchanged graph, a vertex is expanded at most once; repeated encounters with active/done states return immediately. Every outgoing edge of an expanded vertex is examined at most once. Thus time is $O(V + E)$, and auxiliary storage is $O(V)$ for states plus $O(V)$ worst-case active depth. The adjacency lists themselves occupy $O(V + E)$ input storage.

Edge cases and language comparison

An empty graph returns false; a self-loop returns true. Neighbor indices must identify existing vertices. The `checked at()` calls expose invalid indices through `std::out_of_range` instead of silently treating malformed data as a cycle. Python and JavaScript can implement the same three-state algorithm, but their index/container behavior differs. Python lists and JavaScript arrays are not direct substitutes for checked C++ vector access; validation must follow each language's rules.

4 • SYNTHESIS AND DESIGN DECISIONS

Choosing a recursion strategy

Dimension	Python	JavaScript	C++
Example's reduction	n decreases to zero.	k decreases to cached bases.	Finite vertices move through states.
Shared data	No mutable cache.	One Map per outer invocation.	One graph and state table by reference.
Depth	Linear factorial chain.	Linear despite memoization.	Longest DFS path; linear worst case.
Representation	Arbitrary-precision int.	Number indices; BigInt results.	Typed vertices, enum states, vectors.
Primary hazard	Valid input can exceed depth limit.	Cache does not remove deep descent.	Cycles and malformed references require policies.

Tail position is a semantic property

A tail call has no pending result-combination work in its caller. Merely placing a recursive expression after return does not make it tail recursive: factorial's multiplication and Fibonacci's additions are still waiting. Some language implementations optimize eligible tail calls, but portability requires knowing the actual guarantee. Do not rely on such optimization for the three programs in this guide.

Memoization is a result-reuse policy

Memoization is effective when subproblems repeat and a cache key fully identifies the relevant computation. A cache is invalid if omitted mutable inputs can change the answer. It can also consume more memory than recomputation would. A cache of finished answers is different from a visitation table that records active computations; cyclic dependencies may require both concepts.

When an explicit stack is preferable

Use an explicit work stack when input depth is untrusted or when traversal must be paused, resumed, or tightly controlled. Store the information recursive frames would otherwise retain: current item, next child position, partial result, and return phase. Replacing recursion with a loop removes implicit call depth, but does not necessarily remove the algorithm's $O(\text{depth})$ information requirement.

A review checklist

- State the domain, direct answers, and progress measure before coding.
- Identify local state, shared state, and operations waiting after each call.
- Separate total calls, distinct subproblems, active depth, and object storage.
- Test empty/minimal input, invalid input, deep input, repeated subproblems, and cycles.

5 • ASSESSMENT AND CONSOLIDATION

Practice with explanations

Exercises

- **1. Trace:** list every active factorial argument at the deepest point for `factorial(4)`, then give the return sequence. Explain why `n` is not a shared counter.
- **2. Repair a proof:** suppose the recursive factorial call uses `n - 2` while the only base test is `n == 0`. Which positive inputs fail to reach the base? Propose a valid domain or a corrected definition.
- **3. Count work:** for the JavaScript example with input 6, list all cache entries at completion. Explain why counting distinct entries is not identical to counting function invocations.
- **4. Reason about sharing:** would moving Map creation inside solve preserve memoization across sibling calls? Explain using object lifetime and reachability.
- **5. Analyze a graph:** compare edges `0 -> 1`, `0 -> 2`, `1 -> 3`, `2 -> 3` with edges `0 -> 1`, `1 -> 2`, `2 -> 1`. Identify the decisive done or active encounter.
- **6. Design:** outline an iterative DFS frame containing a vertex and a next-neighbor position. State when a vertex becomes active and when it becomes done.

Selected answers and reasoning

- 1.** The active arguments are 4, 3, 2, 1, 0. Results return as 1, 1, 2, 6, 24. Each activation has its own parameter binding. **2.** Odd positive values descend through 1 to -1; the validation raises `ValueError`, so the function no longer computes factorial for that input. Merely adding a base at 1 would instead define a different product recurrence, not ordinary factorial.
- 3.** Keys 0 through 6 exist, with values 0n, 1n, 1n, 2n, 3n, 5n, 8n. Cache-hit calls still count as invocations. **4.** A fresh cache in every solve call prevents siblings from reusing completed results. **5.** The first graph revisits done vertex 3; the second revisits active vertex 1 and contains a cycle.

Common misconceptions

A visible base case does not prove termination. Two recursive calls do not automatically imply exponential complexity. Memoization does not guarantee shallow execution. A syntax tree's repeated function name does not capture every indirect recursive behavior. Stack exhaustion is not evidence that the underlying mathematical recurrence is infinite.

Final synthesis

Recursion connects a definition to an execution model. A strong explanation identifies what each call knows, what it waits for, how it makes progress, and what it retains. The three examples move from numerical reduction to result reuse to finite-state traversal over cycles: the same calling mechanism supports very different correctness arguments.

REFERENCES • FURTHER STUDY

Primary documentation and reading

The programs, traces, and exercises in this guide are original teaching examples. The references below support language-specific details. Language semantics are distinguished from implementation choices; draft specifications may evolve. Links checked 30 September 2026.

[1] Python Software Foundation. `ast` - Abstract Syntax Trees.

FunctionDef, Call, If, and Return nodes; structural analysis of source. [Open reference](#)

[2] Python Software Foundation. `sys` - System-specific Parameters and Functions.

Interpreter recursion limits and the cautions associated with changing them. [Open reference](#)

[3] Ecma International / TC39. ECMAScript: Execution Contexts.

Abstract execution contexts and lexical environments; not prescribed memory layouts. [Open reference](#)

[4] Ecma International / TC39. ECMAScript: The BigInt Type.

The language's integer numeric type used for the Fibonacci results. [Open reference](#)

[5] MDN Web Docs. `InternalError: Too Much Recursion`.

Examples of engine-dependent errors caused by excessive recursion depth. [Open reference](#)

[6] ISO C++ Working Draft. Automatic Storage Duration [`basic.stc.auto`].

Scope-related storage duration of automatic objects and parameters. [Open reference](#)

[7] ISO C++ Working Draft. The Return Statement [`stmt.return`].

Return behavior and destruction associated with leaving a function. [Open reference](#)

[8] Python Software Foundation. Built-in Types: Numeric Types.

Arbitrary-precision Python integers and distinctions among numeric types. [Open reference](#)

Suggested study sequence

Trace the Python example by hand before running it. Add cache-hit logging to the JavaScript example and distinguish calls from new computations. Finally, draw the C++ graphs and label vertices unseen, active, or done as each recursive call begins and ends.

For the next lesson, compare these suspended computations with iteration: which state becomes an accumulator, which state must remain in a work stack, and which resource costs remain unchanged?